

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский университет ИТМО

Факультет программной инженерии и компьютерной техники
Направление подготовки 09.03.01 Программная инженерия
Дисциплина «*Распределенные системы хранения данных*»

Отчет по лабораторной работе №3
Вариант: 100455

Студенты:

Долинный Михаил Владимирович

Антипин Григорий Викторович

группа Р3332

Преподаватель:

Харитоновна Анастасия Евгеньевна

Задание

Цель работы - ознакомиться с методами и средствами построения отказоустойчивых решений на базе СУБД Postgres; получить практические навыки восстановления работы системы после отказа.

Работа рассчитана на двух человек и выполняется в три этапа: настройка, симуляция и обработка сбоя, восстановление.

Требования к выполнению работы

- В качестве хостов использовать одинаковые виртуальные машины.
- В первую очередь необходимо обеспечить сетевую связность между ВМ.
- Для подключения к СУБД (например, через psql), использовать отдельную виртуальную или физическую машину.
- Демонстрировать наполнение базы и доступ на запись на примере **не менее, чем двух** таблиц, столбцов, строк, транзакций и клиентских сессий.

Этап 1. Конфигурация

Развернуть postgres на двух узлах в режиме трансляции логов. Не использовать дополнительные пакеты. Продемонстрировать доступ в режиме чтение/запись на основном сервере. Продемонстрировать, что новые данные синхронизируются на резервный сервер.

Этап 2. Симуляция и обработка сбоя

2.1 Подготовка:

- Установить несколько клиентских подключений к СУБД.
- Продемонстрировать состояние данных и работу клиентов в режиме чтение/запись.

2.2 Сбой:

Симулировать переполнение дискового пространства на основном узле - заполнить всё свободное пространство раздела с PGDATA “мусорными” файлами.

2.3 Обработка:

- Найти и продемонстрировать в логах релевантные сообщения об ошибках.
- Выполнить переключение (failover) на резервный сервер.

- Продemonстрировать состояние данных и работу клиентов в режиме чтение/запись.

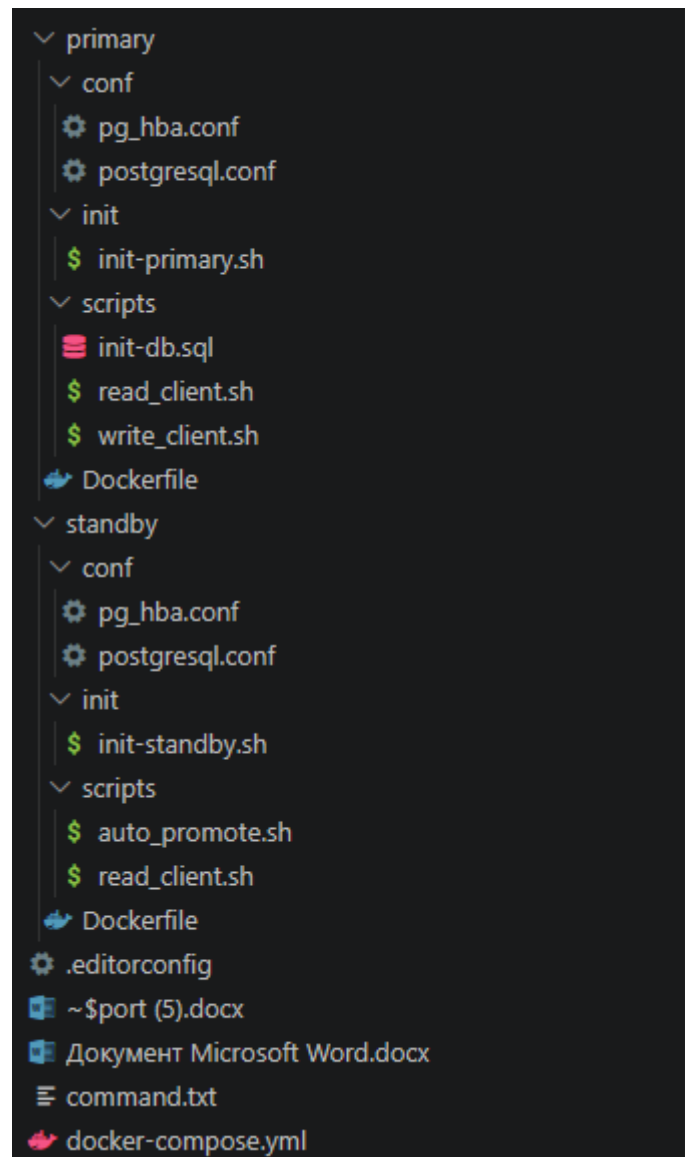
Этап 3. Восстановление

- Восстановить работу основного узла - откатить действие, выполненное с виртуальной машиной на этапе 2.2.
- Актуализировать состояние базы на основном узле - накатить все изменения данных, выполненные на этапе 2.3.
- Восстановить исправную работу узлов в исходной конфигурации (в соответствии с этапом 1).
- Продemonстрировать состояние данных и работу клиентов в режиме чтение/запись.

Выполнение

Подготовка окружения

Структура:



Хосты настроены через docker-compose.yml:

```
services:
  primary:
    container_name: primary
    build:
      context: ./primary
    restart: unless-stopped
    ports:
      - "5432:5432"
    environment:
      - PGDATA=/var/lib/postgresql/data
      - PGENCODING=UTF8
      - PGLOCALE=en_US.UTF8
      - PGUSERNAME=postgres
      - POSTGRES_PASSWORD=postgres
    volumes:
      - /mnt/pgdisk/primary:/var/lib/postgresql/data
    networks:
      - pg_net

  standby:
    container_name: standby
    build:
      context: ./standby
    restart: unless-stopped
    ports:
      - "5433:5432"
    depends_on:
      - primary
    environment:
      - PGDATA=/var/lib/postgresql/data
      - PGENCODING=UTF8
      - PGLOCALE=en_US.UTF8
      - PGUSERNAME=postgres
      - POSTGRES_PASSWORD=postgres
    volumes:
      - /home/goga/standby_data/data:/var/lib/postgresql/data
    networks:
      - pg_net

networks:
  pg_net:
    driver: bridge
```

Соединение между хостами настроено через сеть pg_net и порты 5432 и 5433 соответственно.

Этап 1. Конфигурация

primary

Dockerfile

```
FROM postgres:latest
COPY conf/* /etc/postgresql/
COPY scripts/* /home/scripts/
COPY init/init-primary.sh /docker-entrypoint-initdb.d/10-init-primary.sh
COPY init/init-primary.sh /home/init/init-primary.sh
RUN chmod +x /home/scripts/read_client.sh
RUN chmod +x /home/scripts/write_client.sh
RUN chmod +x /docker-entrypoint-initdb.d/10-init-primary.sh
RUN chmod +x /home/init/init-primary.sh
```

init-primary.sh

```
#!/bin/bash
set -e

if ! psql -U postgres -Atqc "SELECT 1 FROM pg_roles WHERE rolname =
'replicator'" | grep -q 1; then
    psql -v ON_ERROR_STOP=1 --username "postgres" -c "CREATE ROLE replicator
WITH REPLICATION PASSWORD '123' LOGIN;"
fi

if ! psql -U postgres -Atqc "SELECT 1 FROM pg_database WHERE datname =
'losiento'" | grep -q 1; then
    psql -v ON_ERROR_STOP=1 --username "postgres" -f "/home/scripts/init-db.sql"
fi

cp /etc/postgresql/postgresql.conf "$PGDATA/postgresql.conf"
cp /etc/postgresql/pg_hba.conf "$PGDATA/pg_hba.conf"
```

postgresql.conf

```
listen_addresses = '*'
wal_level = replica
wal_keep_size = 64MB
max_wal_senders = 10
max_replication_slots = 10
archive_mode = on
archive_command = 'echo "dummy command, archive_command called"'
log_connections = on
```

```
log_disconnections = on
log_duration = on
wal_log_hints = on
```

pg_hba.conf

#	TYPE	DATABASE	USER	ADDRESS	METHOD
	local	all	all		trust
	host	all	all	0.0.0.0/0	md5
	host	replication	replicator	0.0.0.0/0	md5

standby

Dockerfile

```
FROM postgres:latest
COPY conf/* /etc/postgresql/
COPY scripts/* /home/scripts/
COPY init/init-standby.sh /docker-entrypoint-initdb.d/init-standby.sh
RUN chmod +x /home/scripts/read_client.sh
RUN chmod +x /docker-entrypoint-initdb.d/init-standby.sh
RUN chmod +x /home/scripts/auto_promote.sh
```

init-standby.sh

```
#!/bin/bash
set -e

# Ожидаем готовность основного кластера
until pg_isready -h primary -p 5432 -U postgres; do
    echo "Waiting for primary to be ready..."
    sleep 2
done

# Остановка сервера
pg_ctl stop -D "$PGDATA"

# Отчищаем
rm -rf "$PGDATA"/*
echo "Data directory cleaned up"

# Тасим backup
PGPASSWORD='123' pg_basebackup -h primary -D /var/lib/postgresql/data -U
replicator -v -P --wal-method=stream
echo "Base backup completed"
```

```
# Создаем сигнал
touch "$PGDATA/standby.signal"

# Устанавливаем доступы
chown -R postgres:postgres "$PGDATA"

# Копируем файлы конфигурации
cp /etc/postgresql/postgresql.conf "$PGDATA/postgresql.conf"
cp /etc/postgresql/pg_hba.conf "$PGDATA/pg_hba.conf"
echo "Conf files copied"

# Поднимаем
pg_ctl -D "$PGDATA" start
```

postgresql.conf

```
listen_addresses = '*'
hot_standby = on
primary_conninfo = 'host=primary port=5432 user=replicator password=123'
wal_log_hints = on
log_connections = on
log_disconnections = on
log_duration = on
wal_log_hints = on
```

pg_hba.conf

#	TYPE	DATABASE	USER	ADDRESS	METHOD
	local	all	all		trust
	host	all	all	0.0.0.0/0	md5
	host	replication	replicator	0.0.0.0/0	md5

Запуск

Запуск primary:

```
docker-compose up -d --build primary
```

```
=> => naming to docker.io/library/lab3-primary:latest 0.0s
=> => unpacking to docker.io/library/lab3-primary:latest 0.2s
=> resolving provenance for metadata file 0.0s
[+] up 3/3
✓Image lab3-primary Built 2.2s
✓Network lab3_pg_net Created 0.1s
✓Container primary Started 0.8s
```

Ожидаем лога "database system is ready to accept connections":

```
2026-03-28 14:29:08.526 GMT [1] LOG: starting PostgreSQL 18.3 (Debian 18.3-1.pgdg13+1) on x86_64-pc-
linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
2026-03-28 14:29:08.527 GMT [1] LOG: listening on IPv4 address "0.0.0.0", port 5432
2026-03-28 14:29:08.527 GMT [1] LOG: listening on IPv6 address ":::", port 5432
2026-03-28 14:29:08.535 GMT [1] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
2026-03-28 14:29:08.553 GMT [82] LOG: database system was shut down at 2026-03-28 14:29:08 GMT
2026-03-28 14:29:08.571 GMT [1] LOG: database system is ready to accept connections
```

Запуск standby:

```
sudo docker-compose up -d --build standby
```

```
=> => unpacking to docker.io/library/lab3-primary:latest 0.0s
=> [primary] resolving provenance for metadata file 0.0s
=> [standby] resolving provenance for metadata file 0.0s
[+] up 4/4
✓Image lab3-standby Built 2.0s
✓Image lab3-primary Built 1.9s
✓Container primary Started 1.1s
✓Container standby Started 0.7s
```

Видим что репликатор подключился:

```
2026-03-28 14:36:45.145 GMT [43] LOG: disconnection: session time: 0:00:00.091 user=replicator
database= host=172.18.0.3 port=58270
2026-03-28 14:36:45.306 GMT [44] LOG: connection received: host=172.18.0.3 port=58284
2026-03-28 14:36:45.312 GMT [44] LOG: connection authenticated: identity="replicator" method=md5
(/var/lib/postgresql/data/pg_hba.conf:4)
2026-03-28 14:36:45.312 GMT [44] LOG: replication connection authorized: user=replicator
application_name=walreceiver
```

Наполнение базы

На примере не менее, чем двух таблиц, столбцов, строк, транзакций и клиентских сессий, `init-db.sql`:

```
CREATE DATABASE losiento;

\c losiento;

CREATE TABLE employees (
    id SERIAL PRIMARY KEY,
    full_name TEXT,
    department TEXT
);

CREATE TABLE projects (
    id SERIAL PRIMARY KEY,
    title TEXT,
    budget INTEGER
);

CREATE TABLE tasks (
    id SERIAL PRIMARY KEY,
    project_id INTEGER,
    description TEXT,
    status TEXT
);

INSERT INTO employees (full_name, department)
SELECT
    'Сотрудник ' || g,
    (ARRAY['IT','HR','Sales','Finance'])[1 + (g % 4)]
FROM generate_series(1, 10) g;

INSERT INTO projects (title, budget)
SELECT
    'Проект ' || g,
    g * 10000
FROM generate_series(1, 10) g;

INSERT INTO tasks (project_id, description, status)
SELECT
    1 + (g % 100),
    'Задача номер ' || g,
    (ARRAY['new','in_progress','done'])[1 + (g % 3)]
FROM generate_series(1, 20) g;
```

Проверим данные на primary:

```
goga@goga: /mnt/c/Users/antip/OneDrive/Desktop/butwhy/PCXD/lab3$ sudo docker exec -it primary
psql -U postgres -d losiento -c "SELECT * FROM employees;"
 id | full_name | department
-----+-----+-----
  1 | Сотрудник 1 | HR
  2 | Сотрудник 2 | Sales
  3 | Сотрудник 3 | Finance
  4 | Сотрудник 4 | IT
  5 | Сотрудник 5 | HR
  6 | Сотрудник 6 | Sales
  7 | Сотрудник 7 | Finance
  8 | Сотрудник 8 | IT
  9 | Сотрудник 9 | HR
 10 | Сотрудник 10 | Sales
(10 rows)
```

Проверяем режим записи:

```
sudo docker exec -it primary psql -U postgres -d losiento -c "INSERT INTO
employees (full_name, department) VALUES ('Wooopa', 'IT');"
sudo docker exec -it standby psql -U postgres -d losiento -c "INSERT INTO
employees (full_name, department) VALUES ('Wooopa', 'IT');"

```

```
goga@goga: /mnt/c/Users/antip/OneDrive/Desktop/butwhy/PCXD/lab3$ sudo docker exec -it primary
psql -U postgres -d losiento -c "INSERT INTO employees (full_name, department) VALUES ('Woo
opa', 'IT');"
INSERT 0 1
goga@goga: /mnt/c/Users/antip/OneDrive/Desktop/butwhy/PCXD/lab3$ sudo docker exec -it standby
psql -U postgres -d losiento -c "INSERT INTO employees (full_name, department) VALUES ('Woo
opa', 'IT');"
ERROR:  cannot execute INSERT in a read-only transaction

```

Проверим режим чтения на standby:

```
sudo docker exec -it standby psql -U postgres -d losiento -c "SELECT * FROM
employees;"

```

```
goga@goga: /mnt/c/Users/antip/OneDrive/Desktop/butwhy/PCXD/lab3$ sudo docker exec -it standby
psql -U postgres -d losiento -c "SELECT * FROM employees;"
 id | full_name | department
-----+-----+-----
  1 | Сотрудник 1 | HR
  2 | Сотрудник 2 | Sales
  3 | Сотрудник 3 | Finance
  4 | Сотрудник 4 | IT
  5 | Сотрудник 5 | HR
  6 | Сотрудник 6 | Sales
  7 | Сотрудник 7 | Finance
  8 | Сотрудник 8 | IT
  9 | Сотрудник 9 | HR
 10 | Сотрудник 10 | Sales
 11 | Wooopa      | IT
(11 rows)
```

Этап 2. Симуляция и обработка сбоя

2.1 Подготовка

Скрипты для симуляции чтения и записи из primary и чтение из standby:

read_client.sh

```
#!/bin/bash
while true; do
    psql -U postgres -d losiento -c "SELECT * FROM employees;"
    psql -U postgres -d losiento -c "SELECT * FROM projects;"
    psql -U postgres -d losiento -c "SELECT * FROM tasks;"
    sleep 2
done
```

write_client.sh

```
#!/bin/bash

while true; do
    # 1. Добавляем нового сотрудника
    departments=("IT" "HR" "Sales" "Finance")
    random_department=${departments[$RANDOM % ${#departments[@]}]}
    employee_name="Сотрудник_$(date +%s)"

    psql -U postgres -d losiento -c \
        "INSERT INTO employees (full_name, department) VALUES
('$employee_name', '$random_department');"

    # 2. Добавляем новый проект
    project_title="Проект_$(date +%s)"
    budget=$(( (RANDOM % 90 + 10) * 10000 ))

    psql -U postgres -d losiento -c \
        "INSERT INTO projects (title, budget) VALUES ('$project_title',
$budget);"

    # 3. Берём id последнего проекта
    last_project_id=$(psql -U postgres -d losiento -t -A -c \
        "SELECT id FROM projects ORDER BY id DESC LIMIT 1;")

    # 4. Добавляем задачу к последнему проекту
    statuses=("new" "in_progress" "done")
    random_status=${statuses[$RANDOM % ${#statuses[@]}]}
    task_description="Задача_$(date +%s)"

    psql -U postgres -d losiento -c \
```

```
"INSERT INTO tasks (project_id, description, status) VALUES
($last_project_id, '$task_description', '$random_status');"

sleep 2
done
```

Запуск:

```
sudo docker exec -it primary bash /home/scripts/read_client.sh
sudo docker exec -it standby bash /home/scripts/read_client.sh
sudo docker exec -it primary bash /home/scripts/write_client.sh
```

Ожидаем что в standby чтение будет автоматически показывать новые данные из primary:

id	full_name	department
1	Сотрудник 1	HR
2	Сотрудник 2	Sales
3	Сотрудник 3	Finance
4	Сотрудник 4	IT
5	Сотрудник 5	HR
6	Сотрудник 6	Sales
7	Сотрудник 7	Finance
8	Сотрудник 8	IT
9	Сотрудник 9	HR
10	Сотрудник 10	Sales
11	Woopa	IT
12	Сотрудник_1774709153	Sales
13	Сотрудник_1774709156	Sales
14	Сотрудник_1774709158	Sales
15	Сотрудник_1774709160	Finance

(15 rows)

id	title	budget
1	Проект 1	10000
2	Проект 2	20000
3	Проект 3	30000
4	Проект 4	40000
5	Проект 5	50000
6	Проект 6	60000
7	Проект 7	70000
8	Проект 8	80000
9	Проект 9	90000
10	Проект 10	100000
11	Проект_1774709154	420000
12	Проект_1774709156	210000
13	Проект_1774709158	200000
14	Проект_1774709160	520000

(14 rows)

id	project_id	description	status
1	2	Задача номер 1	in_progress
2	3	Задача номер 2	done
3	4	Задача номер 3	new
4	5	Задача номер 4	in_progress
5	6	Задача номер 5	done

2.2 Сбой

Проверяем права

```
sudo docker exec -it primary bash -lc "psql -U postgres -c 'SELECT pg_is_in_recovery();'"
sudo docker exec -it standby bash -lc "psql -U postgres -c 'SELECT pg_is_in_recovery();'"
```

```
goga@goga:/mnt/c/Users/antip/OneDrive/Desktop/butwhy/PCXD/lab3$ sudo docker exec -it primary
bash -lc "psql -U postgres -c 'SELECT pg_is_in_recovery();'"
sudo docker exec -it standby bash -lc "psql -U postgres -c 'SELECT pg_is_in_recovery();'"
pg_is_in_recovery
-----
f
(1 row)

pg_is_in_recovery
-----
t
(1 row)
```

Проверяем сколько памяти осталось

```
sudo docker exec -it primary bash -lc "df -h /var/lib/postgresql/data"
```

```
goga@goga:/mnt/c/Users/antip/OneDrive/Desktop/butwhy/PCXD/lab3$ sudo docker exec -it primary
bash -lc "df -h /var/lib/postgresql/data"
Filesystem      Size  Used Avail Use% Mounted on
/dev/loop2      3.9G  11M   3.6G   3% /var/lib/postgresql/data
```

Заполняем мусором

```
sudo docker exec -it primary bash -lc "mkdir -p /var/lib/postgresql/data/fill
&& dd if=/dev/zero of=/var/lib/postgresql/data/fill/junk.bin bs=16M
status=progress"
```

```
goga@goga:/mnt/c/Users/antip/OneDrive/Desktop/butwhy/PCXD/lab3$ sudo docker exec -it primary
bash -lc "mkdir -p /var/lib/postgresql/data/fill && dd if=/dev/zero of=/var/lib/postgresql/
data/fill/junk.bin bs=16M status=progress"
3992977408 bytes (4.0 GB, 3.7 GiB) copied, 9 s, 425 MB/s
dd: error writing '/var/lib/postgresql/data/fill/junk.bin': No space left on device
240+0 records in
239+0 records out
4010532864 bytes (4.0 GB, 3.7 GiB) copied, 9.59684 s, 418 MB/s
```

Докидываем остатки памяти insertами и ждем панику

```

INSERT 0 1
INSERT 0 1
ERROR: could not extend file "base/16385/16407": No space left on device
HINT: Check free disk space.
INSERT 0 1
INSERT 0 1
ERROR: could not extend file "base/16385/16407": No space left on device
HINT: Check free disk space.
INSERT 0 1
INSERT 0 1
ERROR: could not extend file "base/16385/16407": No space left on device
HINT: Check free disk space.
INSERT 0 1
INSERT 0 1
ERROR: could not extend file "base/16385/16407": No space left on device
HINT: Check free disk space.
ERROR: could not extend file "base/16385/16387": No space left on device
HINT: Check free disk space.
INSERT 0 1
ERROR: could not extend file "base/16385/16407": No space left on device
HINT: Check free disk space.
ERROR: could not extend file "base/16385/16387": No space left on device
HINT: Check free disk space.
INSERT 0 1
ERROR: could not extend file "base/16385/16407": No space left on device
HINT: Check free disk space.

```

Видим по логам контейнера что основной кластер умер

```

2026-03-28 15:15:11.088 GMT [30] PANIC: could not write to file "pg_logical/replorigin_checkpoint.tmp":
No space left on device
2026-03-28 15:15:11.531 GMT [2577] LOG: connection received: host=[local]
2026-03-28 15:15:11.532 GMT [2577] LOG: connection authenticated: user="postgres" method=trust
(/var/lib/postgresql/data/pg_hba.conf:2)
2026-03-28 15:15:11.532 GMT [2577] LOG: connection authorized: user=postgres database=losiento
application_name=psql
2026-03-28 15:15:11.535 GMT [2577] ERROR: could not extend file "base/16385/16387_fsm": No space left
on device
2026-03-28 15:15:11.535 GMT [2577] HINT: Check free disk space.
2026-03-28 15:15:11.535 GMT [2577] STATEMENT: INSERT INTO employees (full_name, department) VALUES
('Сотрудник_1774710911', 'Finance');
2026-03-28 15:15:11.536 GMT [2577] LOG: disconnection: session time: 0:00:00.004 user=postgres
database=losiento host=[local]
2026-03-28 15:15:11.576 GMT [2580] LOG: connection received: host=[local]
2026-03-28 15:15:11.578 GMT [2580] LOG: connection authenticated: user="postgres" method=trust
(/var/lib/postgresql/data/pg_hba.conf:2)
2026-03-28 15:15:11.578 GMT [2580] LOG: connection authorized: user=postgres database=losiento
application_name=psql
2026-03-28 15:15:11.581 GMT [2580] ERROR: could not extend file "base/16385/16397_fsm": No space left
on device
2026-03-28 15:15:11.581 GMT [2580] HINT: Check free disk space.
2026-03-28 15:15:11.581 GMT [2580] STATEMENT: INSERT INTO projects (title, budget) VALUES
('Проект_1774710911', 540000);

```

☐		Name	Container ID	Image	Port(s)	Actions
☐	✓	lab3	-	-	-	   
☐	↻	primary	9811a69bec56	lab3-primar	5432:5432	   
☐	●	standby	c904f5e350d2	lab3-standb	5433:5432 	   

Делаем failover:

```
sudo docker exec -it standby bash -lc "psql -U postgres -c 'SELECT pg_promote();'"
```

```
goga@goga:/mnt/c/Users/antip/OneDrive/Desktop/butwhy/PCXD/lab3$ sudo docker exec -it standby bash -lc "psql -U postgres -c 'SELECT pg_promote();'"
pg_promote
-----
 t
(1 row)
```

Проверяем:

```
sudo docker exec -it standby bash -lc "psql -U postgres -c 'SELECT pg_is_in_recovery();'"
```

```
sudo docker exec -it standby bash -lc "psql -U postgres -c 'SHOW transaction_read_only;'"
```

```
goga@goga:/mnt/c/Users/antip/OneDrive/Desktop/butwhy/PCXD/lab3$ sudo docker exec -it standby bash -lc "psql -U postgres -c 'SELECT pg_is_in_recovery();'"
sudo docker exec -it standby bash -lc "psql -U postgres -c 'SHOW transaction_read_only;'"
pg_is_in_recovery
-----
 f
(1 row)

transaction_read_only
-----
 off
(1 row)
```

Этап 3. Восстановление

Добиваем primary чтобы не дергался

```
sudo docker stop primary
```

Сносим папку

```
sudo rm -rf /mnt/pgdisk/primary
sudo mkdir -p /mnt/pgdisk/primary
sudo chown -R 999:999 /mnt/pgdisk/primary
```

Заново поднимаем primary на чистую

```
sudo docker compose up -d --build primary
```

Делаем backup со standby

```
sudo docker exec -it primary bash -lc "
rm -rf /tmp/backup &&
mkdir -p /tmp/backup &&
chown -R postgres:postgres /tmp/backup
"

sudo docker exec -it primary bash -lc "
su postgres -c '
PGPASSWORD=123 \
pg_basebackup -h standby \
-D /tmp/backup \
-U replicator \
-Fp -Xs -P -R
'
"
```

```
goga@goga:/mnt/c/Users/antip/OneDrive/Desktop/butwhy/PCXD/lab3$ sudo docker exec -it primary
bash -lc "
rm -rf /tmp/backup &&
mkdir -p /tmp/backup &&
chown -R postgres:postgres /tmp/backup
"
goga@goga:/mnt/c/Users/antip/OneDrive/Desktop/butwhy/PCXD/lab3$ sudo docker exec -it primary
bash -lc "
su postgres -c '
PGPASSWORD=123 \
pg_basebackup -h standby \
-D /tmp/backup \
-U replicator \
-Fp -Xs -P -R
'
"
31543/31543 kB (100%), 1/1 tablespace
```

Вытаскиваем backup на хост

```
sudo rm -rf /tmp/primary_backup_copy
sudo mkdir -p /tmp/primary_backup_copy
sudo docker cp primary:/tmp/backup/. /tmp/primary_backup_copy/
```

```
goga@goga:/mnt/c/Users/antip/OneDrive/Desktop/butwhy/PCXD/lab3$ sudo rm -rf /tmp/primary_backup_copy
goga@goga:/mnt/c/Users/antip/OneDrive/Desktop/butwhy/PCXD/lab3$ sudo mkdir -p /tmp/primary_backup_copy
goga@goga:/mnt/c/Users/antip/OneDrive/Desktop/butwhy/PCXD/lab3$ sudo docker cp primary:/tmp/backup/. /tmp/primary_backup_copy/
Successfully copied 49.3MB to /tmp/primary_backup_copy/
```

Опять сносим primary

```
sudo docker rm -f primary
```

Копируем backup на пустую директорию primary

```
sudo cp -a /tmp/primary_backup_copy/. /mnt/pgdisk/primary/
sudo chown -R 999:999 /mnt/pgdisk/primary
```

И опять собираем primary

```
sudo docker compose up -d --build primary
```

```
=> => naming to docker.io/library/lab3-primary:latest 0.0s
=> => unpacking to docker.io/library/lab3-primary:latest 0.0s
=> resolving provenance for metadata file 0.0s
[+] up 2/2
  ✓Image lab3-primary Built 1.6s
  ✓Container primary Started 0.4s
```

Возвращаем исходные права

```
sudo docker stop standby
sudo docker exec -it primary bash -lc "psql -U postgres -c 'SELECT pg_promote();'"
sudo docker exec -it primary bash -lc "psql -U postgres -c 'SELECT pg_is_in_recovery();'"
```

```
[+] up 2/2
  ✓Image lab3-primary Built 1.9s
  ✓Container primary Started 0.4s
goga@goga:/mnt/c/Users/antip/OneDrive/Desktop/butwhy/PCXD/lab3$ sudo docker stop standby
goga@goga:/mnt/c/Users/antip/OneDrive/Desktop/butwhy/PCXD/lab3$ sudo docker exec -it primary bash -lc "psql -U postgres -c 'SELECT pg_promote();'"
goga@goga:/mnt/c/Users/antip/OneDrive/Desktop/butwhy/PCXD/lab3$ sudo docker exec -it primary bash -lc "psql -U postgres -c 'SELECT pg_is_in_recovery();'"
standby
pg_promote
-----
 t
(1 row)

pg_is_in_recovery
-----
 f
(1 row)
```

Удаляем standby и чистим папку

```
# Удаляем standby
sudo docker rm -f standby

# Сносим
sudo rm -rf /home/goga/standby_data/data
sudo mkdir -p /home/goga/standby_data/data
sudo chown -R 999:999 /home/goga/standby_data/data
sudo chmod 700 /home/goga/standby_data/data
```

Делаем backup на standby

```
sudo docker compose run --rm standby bash

su postgres -c "
PGPASSWORD=123 \
pg_basebackup -h primary \
-D /var/lib/postgresql/data \
-U replicator \
-Fp -Xs -P -R
"

sudo docker compose up -d standby
```

```
goga@goga:/mnt/c/Users/antip/OneDrive/Desktop/butwhy/PCXD/lab3$ sudo docker rm -f standby
standby
goga@goga:/mnt/c/Users/antip/OneDrive/Desktop/butwhy/PCXD/lab3$ sudo rm -rf /home/goga/stand
by_data/data
sudo mkdir -p /home/goga/standby_data/data
sudo chown -R 999:999 /home/goga/standby_data/data
sudo chmod 700 /home/goga/standby_data/data
goga@goga:/mnt/c/Users/antip/OneDrive/Desktop/butwhy/PCXD/lab3$ sudo docker compose run --rm
standby bash
[+] 1/1t 1/11
✓Container primary Running 0.0s
Container lab3-standby-run-2aa9346fbccd Creating
Container lab3-standby-run-2aa9346fbccd Created
root@b714dfa756cb:/# su postgres -c "
PGPASSWORD=123 \
pg_basebackup -h primary \
-D /var/lib/postgresql/data \
-U replicator \
-Fp -Xs -P -R
"
31543/31543 kB (100%), 1/1 tablespace
root@b714dfa756cb:/# exit
exit
goga@goga:/mnt/c/Users/antip/OneDrive/Desktop/butwhy/PCXD/lab3$
```

```
goga@goga:/mnt/c/Users/antip/OneDrive/Desktop/butwhy/PCXD/lab3$ sudo docker compose up -d st
andby
[+] up 2/2
✓Container primary Running 0.0s
✓Container standby Started 0.4s
```

И проверяем что все роли на месте

```
sudo docker exec -it primary bash -lc "psql -U postgres -c 'SELECT
pg_is_in_recovery();'"
```

```
sudo docker exec -it standby bash -lc "psql -U postgres -c 'SELECT pg_is_in_recovery();'"
```

```
goga@goga:/mnt/c/Users/antip/OneDrive/Desktop/butwhy/PCXD/lab3$ sudo docker exec -it primary  
bash -lc "psql -U postgres -c 'SELECT pg_is_in_recovery();'"  
sudo docker exec -it standby bash -lc "psql -U postgres -c 'SELECT pg_is_in_recovery();'"  
pg_is_in_recovery  
-----  
f  
(1 row)  
  
pg_is_in_recovery  
-----  
t  
(1 row)
```

Проверяем что все данные на месте:

```
goga@goga:/mnt/c/Users/antip/OneDrive/Desktop/butwhy/PCXD/lab3$ sudo docker exec -it primary  
psql -U postgres -d losiento -c "SELECT COUNT(*) FROM employees;"  
count  
-----  
117  
(1 row)  
  
goga@goga:/mnt/c/Users/antip/OneDrive/Desktop/butwhy/PCXD/lab3$ sudo docker exec -it standby  
psql -U postgres -d losiento -c "SELECT COUNT(*) FROM employees;"  
count  
-----  
117  
(1 row)
```

Вывод

В ходе лабораторной работы была настроена потоковая репликация PostgreSQL между основным и резервным узлами, а также отработаны сценарии отказа и восстановления системы. Были получены практические навыки настройки репликации, выполнения failover и восстановления исходной конфигурации.

Симуляция сбоя показала, что при отказе основного узла система может продолжить работу за счёт резервного, сохраняя целостность данных. Таким образом, цель работы достигнута — изучены методы обеспечения отказоустойчивости и восстановления СУБД PostgreSQL.